

AI reference

使用部分

基本上這個專案大部分都是以 ChatGPT 生成架構，自己再人工做細部修改與 debug，並且有自己 trace code 了解原理與語法，所以底下就附上幾個比較複雜的段落。

ChatroomPage.jsx

```

1  /* 發送 / 收回 */
2  const sendMsg = async () => {
3    if (!newMsg.trim()) return;
4    const newRef = await push(ref(database, `chatrooms/${roomId}/messages`),
5      {uid: user.uid, text: newMsg.trim(), timestamp: Date.now()}
6    );
7    setNewMsg(''); setLastSentKey(newRef.key);
8  };
9  const recall = key => remove(ref(database, `chatrooms/${roomId}/messages/${key}`));
10
11  const toggleBlock = uid => {
12    if (uid === user.uid) return alert('不能封鎖自己!');
13    const upd = blocked.includes(uid) ? blocked.filter(i => i !== uid) : [...blocked, uid];
14    setBlocked(upd);
15    localStorage.setItem('blockedUsers', JSON.stringify(upd));
16    setShowModal(false);
17  };
18
19  const shown = messages.filter(m => m.text.toLowerCase().includes(searchText));
20
21  const handleInvite = async () => {
22    const name = inviteName.trim();
23    if (!name) return alert('請輸入 username');
24    if (name === profiles[user.uid]?.username) return alert('不能邀請自己');
25
26    const snapshot = await get(ref(database, 'profiles'));
27    const allProfiles = snapshot.val() || {};
28
29    const matchedUid = Object.keys(allProfiles).find(
30      uid => allProfiles[uid].username === name
31    );
32
33    if (!matchedUid) {
34      return alert('找不到該 username');
35    }
36
37    if (matchedUid === user.uid) {
38      return alert('不能邀請自己');
39    }
40
41    const memberRef = ref(database, `chatrooms/${roomId}/members/${matchedUid}`);
42    const memberSnap = await get(memberRef);
43    if (memberSnap.exists()) {
44      return alert('此用戶已在聊天室中');
45    }
46
47    await set(memberRef, true);
48    alert('已成功邀請該用戶!');
49    setInviteName('');
50  };
51
52  return (
53    <>
54    {/* --- 邀請用戶欄 --- */}
55    <div style={{ display: 'flex', gap: 10, marginBottom: 10 }}>
56      <input
57        className="input"
58        placeholder="輸入使用者名稱以邀請"

```

```

60     value={inviteName}
61     onChange={e => setInviteName(e.target.value)}
62   />
63   <button className="button" style={{ flexShrink: 0, width: 90 }} onClick={invite}
64     邀請
65   </button>
66 </div>
67
68   {/* 搜尋 */}
69   <input className="input" placeholder="搜尋訊息..." value={searchText}
70     onChange={e => setSearchText(e.target.value)} />
71
72   {/* 訊息清單 */}
73   <div className="message-list">
74     {shown.map(m => {
75       const prof = profiles[m.uid] || {};
76       const me = m.uid === user.uid;
77       const mute = blocked.includes(m.uid);
78
79       return (
80         <div key={m.key}
81           className={`message-item ${m.key === lastSentKey ? 'super-a' : ''}`}
82           style={{ opacity: mute ? 0.5 : 1, color: mute ? '#888' : '#000' }}
83           onMouseEnter={() => setHoverKey(m.key)}
84           onMouseLeave={() => setHoverKey(null)}>
85           {/* 頭貼 */}
86           <div className="avatar"
87             style={{ background: `url(${prof.pfp || ''}) center/cover` }}
88             onClick={() => { setModalProfile({ ...prof, uid: m.uid }) }}
89           </div>
90           {/* 文字 */}
91           <div className="msg-body">
92             <div style={{ fontWeight: 'bold', cursor: 'pointer' }}
93               onClick={() => { setModalProfile({ ...prof, uid: m.uid }) }}
94             {prof.username || '使用者'}
95             </div>
96             <div>{mute ? ' (已屏蔽)' : m.text}</div>
97             <div style={{ fontSize: '0.8em', color: '#666' }}>
98               {new Date(m.timestamp).toLocaleString()}
99             </div>
100           </div>
101           {/* 收回 */}
102           {me && hoverKey === m.key && (
103             <button className="recall-btn" onClick={() => recall(m.key)}>
104               收回
105             </button>
106           )}
107         </div>
108       );
109     })}
110   <div ref={bottomRef}></div>
111 </div>

```

ChatAppPage.jsx

```

1  /* 建立聊天室 */
2  const createRoom = async () => {
3    const name = roomName.trim();
4    if (!name) { alert('請輸入聊天室名稱'); return; }
5    const newRef = await push(ref(database, 'chatrooms'), {
6      name,
7      owner: user.uid,
8      members: { [user.uid]: true }
9    });
10   setRoomName('');
11   setRoomID(newRef.key);
12 };

```

firebase realtime database setting

```

1  {
2    "rules": {
3      "chatrooms": {
4        ".read": "auth != null",
5
6        "$roomId": {
7          ".read": "auth != null && data.child('members').hasChild(auth.uid)",
8
9          ".write": "auth != null && (data.child('members').hasChild(auth.uid)
10
11          "messages": {
12            ".read": "auth != null && root.child('chatrooms').child($roomId).ch
13            ".write": "auth != null && root.child('chatrooms').child($roomId).c
14          },
15
16          "members": {
17            "$uid": {
18              ".write": "auth != null && root.child('chatrooms').child($roomId)
19            }
20          }
21        }
22      },
23
24      "profiles": {
25        ".read": true,
26
27        "$uid": {
28          ".write": "auth != null && auth.uid === $uid"
29        }
30      }
31    }
32  }
33

```